

01 · Speicherphysik & RAM-Typen

Mikrocomputertechnik 3 · DHBW Stuttgart

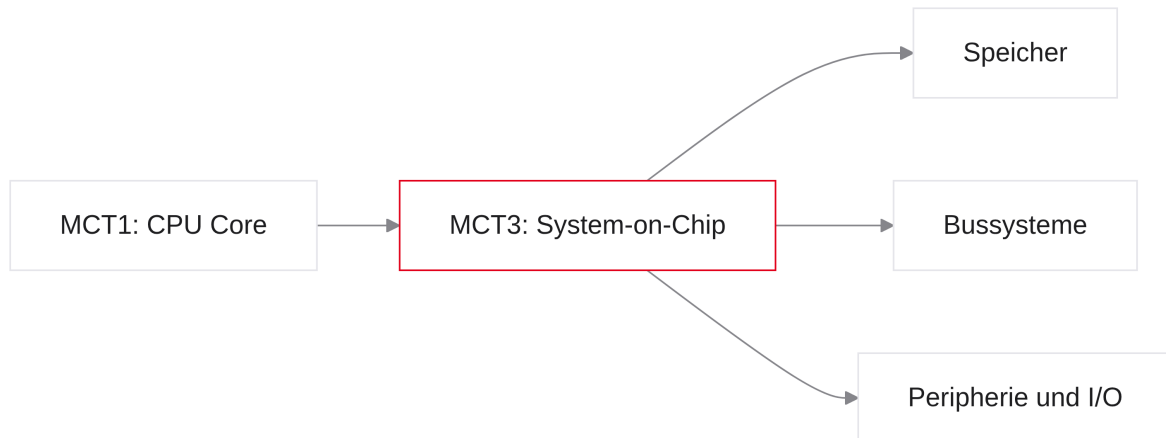
Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 88

Willkommen zu Mikrocomputertechnik 3!

- von der isolierten CPU zum Gesamtsystem
- System-on-Chip (SoC): CPU, Speicher, Busse und Peripherie im Zusammenspiel
- Hardware trifft Software: von nacktem Assembler zur hardwarenahen C-Programmierung



Notizen

Herzlich willkommen zu Mikrocomputertechnik 3! Ein Prozessor allein kann nichts bewirken, wenn er keine Daten speichern oder mit der Außenwelt kommunizieren kann. In diesem Semester bauen wir gedanklich ein komplettes System-on-Chip (SoC) um einen RISC-V-Kern herum. Wir werden C-Code schreiben, der direkt mit der Hardware spricht.

Rückblick: Was wir aus MCT1 wissen

- Die RISC-V ISA: eleganter, modularer Befehlssatz (RV32I)
- Die Register-Werkbank: 32 Architektur-Register als schneller, aber winziger Arbeitsbereich
- Der Datenpfad: Fetch, Decode, Execute, Memory und Write-Back
- Pipelining: mehrere Befehle gleichzeitig auf dem Fließband
- Memory-Mapped I/O: Geräte tarnen sich als Adressen im Arbeitsspeicher

```
# MCT1 Refresher: Basisadresse und I/O-Register
li    s1, 0xF0000000
lw    t0, 0(s1)
andi  t1, t0, 1
beq   t1, zero, Wait
```

Notizen

Erinnern wir uns kurz an unser Handwerkszeug aus MCT1. Wir kennen die Architektur, wir wissen, was ein Load- und ein Store-Befehl auf dem Silizium auslösen und wie Pipelining den Durchsatz maximiert. Am Ende von MCT1 haben wir bereits gesehen, dass LEDs und Schalter über Memory-Mapped I/O angesteuert werden können. Genau an dieser Stelle setzen wir an: Wenn alles über Speicheradressen läuft, müssen wir verstehen, wie diese Speicher physikalisch aufgebaut sind.

Der Paradigmenwechsel: Core vs. SoC

- Der Core (CPU): reines Rechenwerk (ALU, Register, Pipeline)
- Früher: CPU, RAM und I/O als getrennte Bausteine auf der Platine
- Heute: System-on-Chip — alles auf einem Silizium-Die
- Vorteile: weniger Strom, Kosten und Platz; Herausforderung: Systembus

Notizen

Der Begriff Mikroprozessor ist heute eigentlich fast überholt; wir sprechen meistens von Mikrocontrollern oder System-on-Chips. Warum? Weil es aus physikalischen und finanziellen Gründen viel sinnvoller ist, den Arbeitsspeicher, den Festspeicher und die Peripherie-Controller direkt mit der CPU auf dasselbe Stück Silizium zu drucken. Das spart Latenz und Energie, da keine Signale über kupferne Leiterbahnen einer Platine geschickt werden müssen.

SoC: Komponenten am Systembus

Schematischer MCU-SoC — DRAM nicht zwingend auf demselben Die

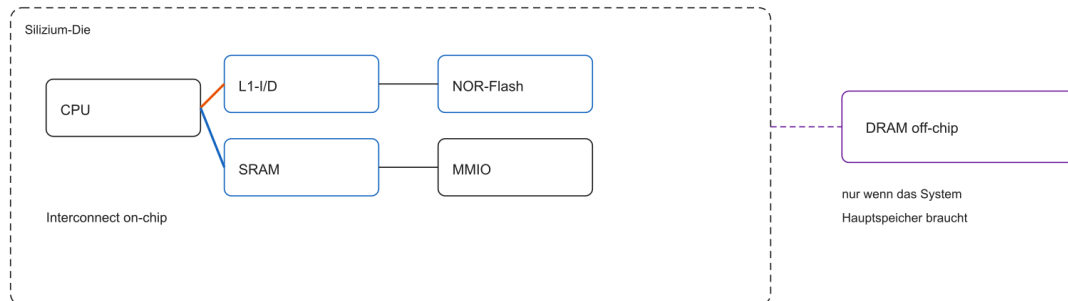


Abbildung 1: On-Chip: CPU, getrennte L1-Pfade, SRAM, NOR-Flash und MMIO. DRAM nur als optionales Off-Chip-Bauteil.

- Cache hängt am Kern, nicht „irgendwo am Bus“ wie ein UART
- NOR-Flash und SRAM können on-chip sein; großer DRAM meist eigener Chip
- Nicht jeder SoC trägt den gesamten Hauptspeicher auf demselben Die

Notizen

Die Abbildung trennt L1 von SRAM/Flash/MMIO und zeichnet DRAM gestrichelt außerhalb des Dies. Ein Sensor-MCU hat oft gar kein DRAM.

Unsere Werkzeuge in MCT3

- Hochsprache statt Assembler: Programmierung in C (Embedded Systems)
- Toolchain: RISC-V GCC übersetzt C in Maschinencode
- Simulator: von Venus zu Renode
- Fokus: Hardware-Treiber statt Sortier-Algorithmen
- Bare-Metal: direkt auf der Hardware, ohne Betriebssystem



Notizen

Da wir nun komplexe Systeme bauen, reicht uns nackter Assembler nicht mehr — die Produktivität wäre zu gering. Wir nutzen C. C ist der Industrie-Standard für hardwarenahe Programmierung, da es uns erlaubt,

Adressen als Pointer direkt anzufassen, während uns der Compiler die mühsame Register-Zuweisung abnimmt. Als Laborumgebung nutzen wir Renode, einen professionellen Open-Source-Simulator, der komplette SoCs simulieren kann.

Folie

6/88

Was ist Renode?

- Industrie-Standard: u. a. bei Google, Intel und Microchip zur Chip-Simulation
- Vollständige SoC-Simulation: CPU, Busse, Timer und RAM
- Professionelles Kommandozeilen-Tool (ohne grafisches Breadboard)
- Funktionale Emulation von CPU, Bussen und Peripherie — kein Versprechen taktgenauer Chip-Nachbildung
- Virtuelle Zeit des Simulators ist nicht dasselbe wie ein HDL-Timing-Modell
- Skript-gesteuert: SoCs über Textdateien (`.resc` / `.repl`)

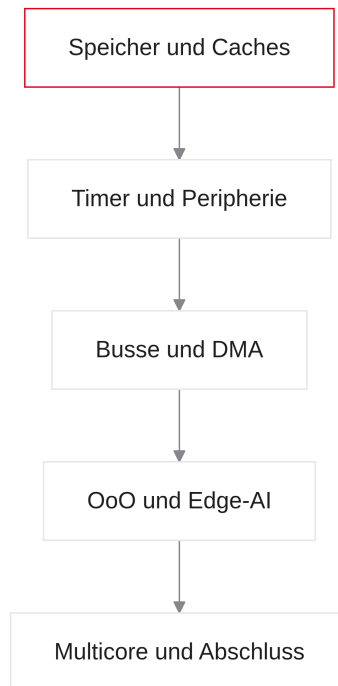
```
# Beispielhaftes Renode Setup-Skript
machine LoadPlatformDescription @platforms/cpus/riscv32.repl
sysbus LoadELF @firmware.elf
start
```

Notizen

Renode eignet sich, um Firmware gegen eine beschriebene Memory Map und Peripherie zu prüfen. Die virtuelle Zeit steuert die funktionale Simulation; taktgenaue Nachbildung eines konkreten Siliziums braucht ein eigenes Timing-Modell oder HDL-Co-Simulation. Im Repository liegt keine `.repl`-Plattform — Adressen in den Folien bleiben deshalb schematisch.

Modul-Übersicht: Unser Fahrplan

- Block 1 & 2: Speicherphysik, RAM-Typen, Caches und die Memory Wall
- Block 3 & 4: Interrupts, Hardware-Timer, PWM und Watchdogs
- Block 5 & 6: Serielle Busse (UART, I2C, SPI) und DMA
- Block 7 & 8: Out-of-Order Execution und Edge-AI (KI-Beschleuniger)
- Block 9: Multicore und Abschluss-Labor



Notizen

Unser Semester ist logisch gegliedert. Wir starten ganz innen im Chip bei den Speichern und Caches. Dann arbeiten wir uns langsam nach außen: über die System-Timer hin zu den externen Bussen, mit denen wir Sensoren auslesen. In der letzten Phase des Semesters widmen wir uns den hochmodernen Konzepten: Out-of-Order CPUs und maschinelles Lernen direkt auf dem Mikrocontroller (Edge AI).

Prüfungsleistung und Laborbetrieb

- Theorie und Praxis: je Einheit 2 Stunden Theorie und 2 Stunden Labor
- Labor als Schlüssel: C-Treiber für das Verständnis der SoC-Architektur
- Prüfungsform: schriftliche Klausur am Semesterende (ca. 90 Minuten)
- Klausurinhalte: C-Code, Pointer-Arithmetik, Architektur, Cache-Berechnungen
- Fokus auf Problemlösung, nicht auf Auswendiglernen von Datenblättern

Notizen

Wie in MCT1 gilt auch hier: Die Vorlesung gibt Ihnen die Konzepte, aber das echte Verständnis kommt im Labor. Wer die Laborblätter selbstständig in Renode löst, hat bereits 80 % der Klausurvorbereitung erledigt. In der Klausur werden wir keine Registeradressen eines bestimmten Chips abfragen, sondern Konzepte: Wie richten Sie einen DMA-Transfer ein? Warum tritt ein Cache-Miss auf?

Agenda für den heutigen Tag

1. Systemarchitektur (Von-Neumann vs. Harvard) und Memory Wall
2. Speicher-Klassifikationen (volatil, non-volatil, Random Access)
3. Festwertspeicher (ROM, EPROM, Flash) — Architektur und Nutzung
4. Volatiler Speicher (SRAM vs. DRAM) — Transistoren vs. Kondensatoren
5. Spezia Speicher im SoC (DPRAM, FIFOs) und Brückenschlag zum Labor

Notizen

Unser heutiges Ziel ist es, das Konzept „Arbeitsspeicher“, das wir in MCT1 als magische Blackbox betrachtet haben, physikalisch zu öffnen. Wir klären heute ein für alle Mal den Unterschied zwischen dem Speicher, auf dem unser Code liegt, und dem Speicher, in dem unsere Variablen leben.

Lernziele Block 1

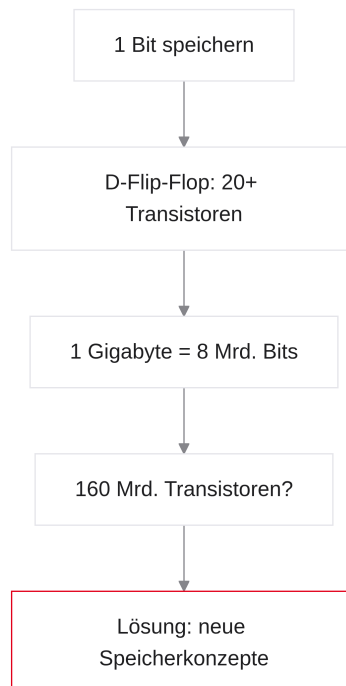
- Memory Wall und Speicherhierarchie-Idee motivieren
- Volatil vs. non-volatil sowie Random Access vs. sequentiell einordnen
- NOR-Flash/XIP vs. NAND und Wear Leveling erklären
- SRAM (6T) vs. DRAM (1T1C, Refresh) unterscheiden
- Memory Map und Linker-Sections (`.text`, `.data`, `.bss`) zuordnen

Notizen

Harris & Harris, *Digital Design and Computer Architecture — RISC-V Edition*, Kap. Memory Systems — Terminologie und Aufbau.

Die Leitfrage des Tages

- Die CPU ist schnell: Milliarden Berechnungen pro Sekunde
- Datenhunger: jede Berechnung braucht Befehle und Daten
- Masse: moderne Programme bestehen aus Milliarden von Bits
- Flip-Flop-Problem: Milliarden D-Flip-Flops sind zu groß, teuer und stromhungrig
- Leitfrage: Wie speichert Silizium Milliarden Bits dicht und effizient?



Notizen

Das D-Flip-Flop aus MCT1 ist großartig, um 32 Register zu bauen. Aber wenn wir ein Gigabyte Speicher bauen wollen und für jedes Bit 20 Transistoren brauchen, würde unser Chip so groß wie ein Fußballfeld werden und sofort verglühen. Wir brauchen grundlegend andere physikalische Konzepte, um Daten dicht und effizient zu speichern. Das ist die Essenz der heutigen Vorlesung.

Folie

12 / 88

Systemarchitektur: Von-Neumann

- Klassisches Modell nach John von Neumann (1945)
- Gemeinsamer Speicher: Befehle und Daten im selben Arbeitsspeicher
- Gemeinsamer Bus für Adressen und Daten zwischen CPU und RAM
- Von-Neumann-Flaschenhals: kein paralleles Laden von Befehl und Variable
- Folge: Structural Hazards auf dem Fließband

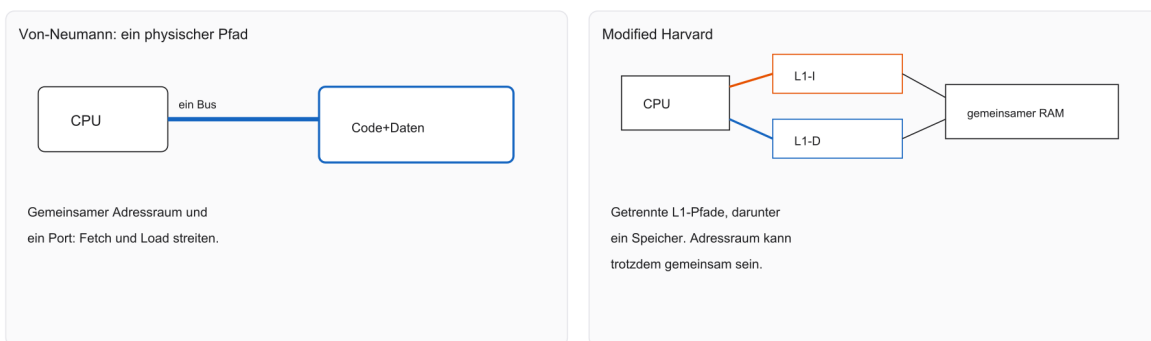


Abbildung 2: Links: ein physischer Port für Code und Daten. Rechts: getrennte L1-I/D, darunter gemeinsamer Speicher.

- Gemeinsamer Adressraum heißt nicht automatisch ein gemeinsamer physischer Port
- Am einfachen Ein-Port-Modell kollidieren Fetch und Load

Notizen

Linkes Panel: ein Bus. Fetch und Datenzugriff können nicht gleichzeitig. Rechtes Panel ist Modified Harvard und gehört zur nächsten Folie.

Systemarchitektur: Harvard

- Harvard-Architektur trennt die Speicherräume strikt
- Getrennte Speicher für Befehle (Code) und Daten
- Getrennte Bus-Systeme zur CPU
- Vorteil: Fetch und Memory im selben Taktzyklus möglich
- Modified Harvard: extern Von-Neumann, intern getrennte L1-Caches

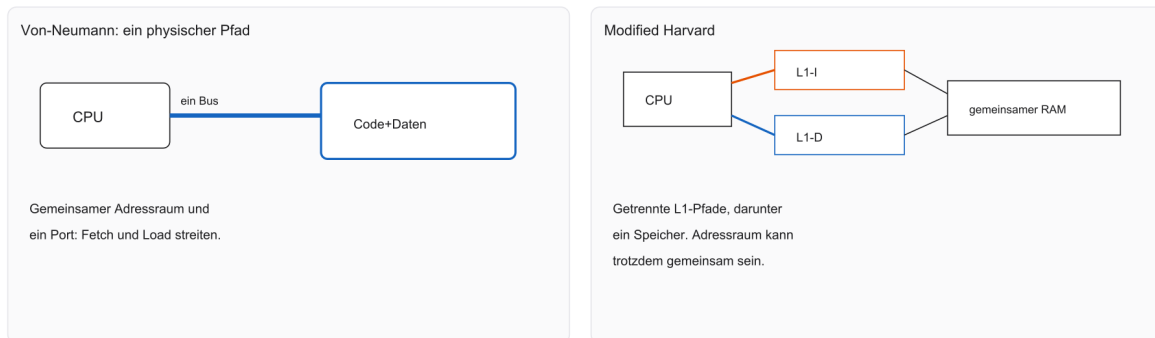


Abbildung 3: Rechtes Panel: L1-I und L1-D getrennt, darunter ein gemeinsamer Speicher.

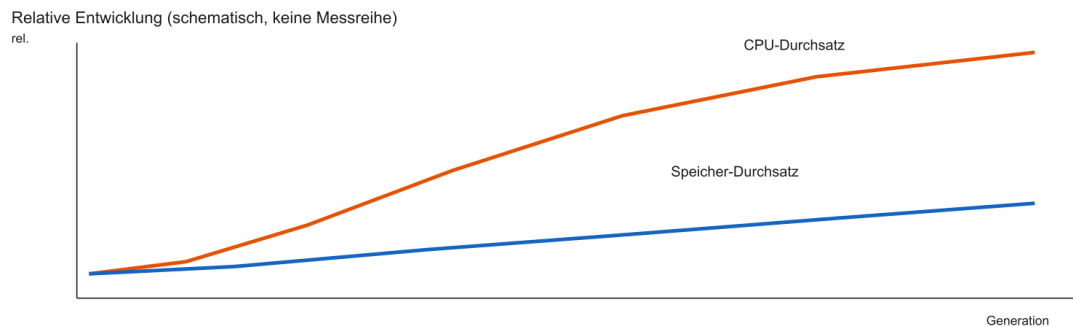
- Striktes Harvard: getrennte Speicher *und* getrennte Busse
- Modified Harvard: getrennte L1-Caches, gemeinsamer Speicher darunter
- Der Adressraum kann trotzdem ein linearer 32-Bit-Raum bleiben

Notizen

MCT1 nutzte getrennte Instruction- und Data-Memories. Viele SoCs sehen von außen wie Von-Neumann aus und entzerren den Kern nur in den L1-Caches — das rechte Panel.

Die Memory Wall

- Moores Gesetz: Transistoren kleiner, CPUs schneller
- Ungleichgewicht: RAM-Geschwindigkeit hielt nicht mit
- Kenngröße hier: Durchsatz (Arbeit pro Zeit), nicht Latenz mit Bandbreite vermischt
- Voll abhängig, ohne Überlappung: $4 \text{ GHz} \times 100 \text{ ns} = 400 \text{ Takte}$



Kenngröße: Durchsatz, nicht Latenz gemischt mit Bandbreite.

Beispiel ohne Überlappung: $4 \text{ GHz} \times 100 \text{ ns} = 400 \text{ Takte Wartezeit}$.

Abbildung 4: Zwei schematische Durchsatz-Kurven: CPU steigt deutlich stärker als die Rate der Speicherzugriffe.

Notizen

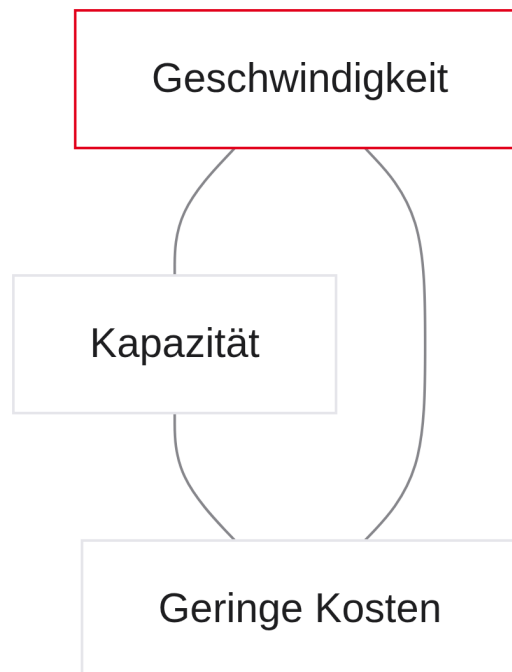
Die Kurven sind keine Messreihe. Die 400 Takte gelten nur für einen Zugriff, der die Pipeline vollständig blockiert und sich nicht mit anderen Befehlen überlappt.

Das magische Speicher-Dreieck

Drei Ziele, die sich gegenseitig ausschließen:

1. Geschwindigkeit — wie schnell lesen/schreiben?
2. Kapazität — wie viele Gigabyte auf dem Chip?
3. Kosten — wie teuer ist ein Bit in der Herstellung?

Eiserne Regel als Modell: Geschwindigkeit, Kapazität und geringe Kosten lassen sich nicht alle drei gleichzeitig maximieren. Beispiel: schnell und groß ist extrem teuer (SRAM); groß und billig ist langsam (Massenspeicher).



Notizen

Dies ist das magische Dreieck der Speicherarchitektur. Wenn jemand zu Ihnen kommt und einen Speicher fordert, der rasend schnell ist, Terabytes an Daten fasst und nur 5 Euro kostet, müssen Sie ihn enttäuschen. Die Physik lässt das nicht zu. Deshalb bestehen Computer nicht aus einem einzigen Speichertyp, sondern aus einer Mischung verschiedener Technologien.

Klassifikation 1: Volatil vs. Non-Volatil

Volatiler Speicher (flüchtig)

- Verliert Daten sofort ohne Betriebsspannung
- Beispiele: SRAM, DRAM, CPU-Register
- Vorteil: meist schnell und beliebig oft beschreibbar

Non-volatiler Speicher (nicht-flüchtig)

- Behält Daten Jahre ohne Strom
- Beispiele: Flash, SD-Karten, ROM
- Nachteil: oft langsamer beim Schreiben, physischer Verschleiß

Notizen

Die grundlegendste Unterscheidung von Speichern ist die Frage nach dem Stromausfall. Volatile Speicher sind das Kurzzeitgedächtnis des Computers. Ziehen Sie den Stecker, ist alles weg. Non-Volatile Speicher sind das Langzeitgedächtnis. Für ein funktionierendes System (wie unseren SoC) brauchen wir zwingend beides: einen Festspeicher für Firmware bzw. Boot-Code und einen Arbeitsspeicher für die laufenden Berechnungen.

Klassifikation 2: Random Access vs. sequenziell

Random Access (wahlfreier Zugriff)

- Jede Adresse ist direkt ansprechbar — die Latenz muss deshalb nicht für jede Anfrage gleich sein
- Beispiel RAM: kein Spulen wie bei einem Band; Caches und DRAM-Zeilen ändern die konkrete Wartezeit trotzdem

Sequenzieller Zugriff

- Daten nur der Reihe nach (wie eine Kassette)
- Ende erreicht man erst nach dem Durchlaufen der davorliegenden Daten

Architektur-Bedeutung: CPUs führen Code nur aus Random-Access-Speichern direkt aus.

Notizen

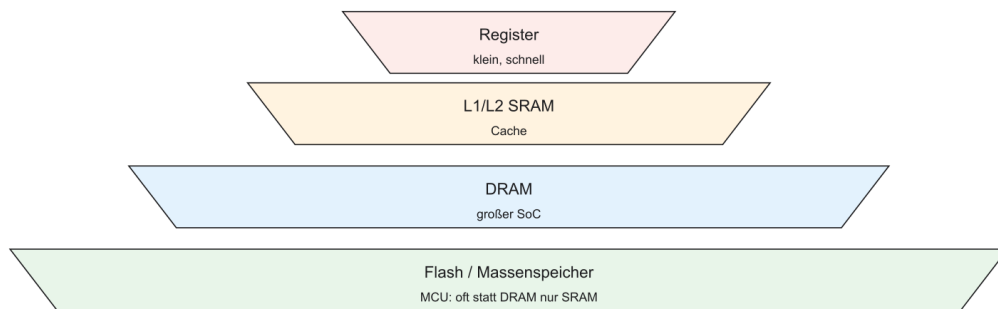
Der Begriff „RAM“ (Random Access Memory) wird oft synonym für Arbeitsspeicher genutzt, aber eigentlich beschreibt er eine Zugriffsmethode. „Wahlfrei“ bedeutet, die Latenz ist unabhängig vom Ort der Daten. Wenn Sie ein Magnetband haben, dauert es Minuten, um an das Ende des Bandes zu spulen. Eine CPU braucht aber für jeden Befehl im Speicher sofortigen Zugriff, daher muss Code zwingend in einem Random-Access-Speicher liegen.

Die Speicherhierarchie-Pyramide

Um die Memory Wall zu überwinden, stapeln Architekten Speicher übereinander:

- Spitze: CPU-Register (klein, teuer, blitzschnell)
- Ebene 2: Caches L1/L2/L3 (SRAM — Megabytes, sehr schnell)
- Ebene 3: Main Memory (DRAM — Gigabytes, moderat)
- Basis: Massenspeicher (Flash/SSD — Terabytes, langsam, billig, non-volatil)

Prinzip: häufig genutzte Daten liegen weiter oben.



↑ Zugriffszeit, Kosten/Bit nach oben kleiner werdende Kapazität

MCU-Beispiel: Flash + SRAM. Großer SoC: Cache, DRAM, Massenspeicher.

Zeiten in der Vorlesung nur als Größenordnung, nicht austauschbare Ebenen.

Abbildung 5: Gestufte Ebenen. Oben weniger Kapazität, kürzere Zugriffszeit, höhere Kosten pro Bit. MCU oft nur Flash+SRAM.

Notizen

Die Pyramide ersetzt die schmale Kästchenkette. Nanosekunden-Angaben der alten Folie vermischten Zellzeit, Cache-Hit und DRAM-Zugriff — die Abbildung bleibt qualitativ.

Fokus heute: Wir öffnen die Chips

- Heute: Physik statt Betriebssystem
- Wir öffnen die Boxen der Speicherhierarchie
- Wie funktioniert Festwertspeicher (ROM/Flash)?
- Warum nutzen SRAM und DRAM andere physikalische Konzepte?
- Was bedeutet das für uns als C-Programmierer am SoC?

Notizen

Nachdem wir nun die Motivation und die Einordnung im System haben, greifen wir uns heute zwei fundamentale Speicherarten heraus: den Festspeicher, auf dem unsere Firmware liegt, und den volatilen Arbeitsspeicher. Wir werden sehen, dass beide extrem unterschiedliche Kompromisse beim Design eingehen mussten.

Folie

20 / 88

Festwertspeicher (Non-Volatile)

- Start unten in der Pyramide: Speicher, der Stromausfall überlebt
- Überbegriff: ROM (Read-Only Memory)
- Historisch wirklich nur lesbar
- Evolution: Firmware nachträglich updatebar machen
- Weg: Mask-ROM, PROM, EPROM, EEPROM, Flash



Abbildung 6: Vergleichsleiste: Programmier- und Löschmethode, ohne Bauteilfotos.

Notizen

Beginnen wir mit dem Langzeitgedächtnis. Jeder Computer muss beim Einschalten wissen, was er tun soll. Er braucht Boot-Code. Dieser Code muss zwingend auf einem nicht-flüchtigen Speicher liegen. Die Geschichte dieses Speichers ist eine faszinierende Reise davon, wie Hardware immer flexibler und softwareähnlicher wurde.

Folie

21 / 88

Mask-ROM

- Nullen und Einsen bei der Herstellung physisch verdrahtet (Fotolithografie-Masken)
- Vorteil: extrem billig in der Massenproduktion
- Nachteil: Bug $\Rightarrow$ alle Chips unbrauchbar
- Früher: Game-Cartridges, Spieluhren
- Heute: kaum noch für Anwendercode; ggf. unveränderliche Boot-Loader

Notizen

Der Mask-ROM ist Hard-Coded im wahrsten Sinne des Wortes. Eine Eins ist eine physische Kupfer-

verbindung, eine Null ist eine durchtrennte Leitung. Wenn Sie als Programmierer einen Fehler gemacht haben, mussten Sie den Halbleiter-Hersteller anrufen, eine neue Belichtungsmaske für 100.000 Dollar bestellen und alle produzierten Chips entsorgen. Die Industrie verlangte dringend nach einer besseren Lösung.

Folie

22 / 88

PROM (Programmable ROM)

- Chip verlässt das Werk leer (alles Einsen) und wird beim Kunden beschrieben
- Jede Zelle besitzt eine mikroskopische Sicherung (Fuse)
- Hohe Spannung „brennt“ die Sicherung durch: 1 → 0
- OTP (One-Time Programmable): einmal durchgebrannt, nie reparierbar
- Gut für kleine Serien; Firmware-Updates weiterhin unmöglich

Notizen

Das PROM war der erste Schritt zur Flexibilität. Man kauft leere Chips und brennt sein Programm mit einem speziellen Programmiergerät hinein. Das passiert wörtlich: Man jagt eine hohe Spannung durch den Chip und brennt winzige Sicherungen durch. Das ist großartig für Kleinserien, aber „One-Time Programmable“ heißt: Bei einem Bug müssen Sie zwar nicht die Maske ändern, aber den Chip müssen Sie trotzdem auslöten und wegwerfen.

Folie

23 / 88

EPROM (Erasable PROM)

- Speicher, der gelöscht und neu beschrieben werden kann
- Quarzglas-Fenster auf der Chip-Oberseite
- UV-Licht (ca. 20–30 Minuten) löscht den gesamten Chip
- Nachteil: immer ganzer Chip, langsam, Ausbau nötig
- Fenster nach dem Brennen abdecken, sonst löscht Sonnenlicht den Code

Notizen

Wenn Sie heute alte Arcade-Automaten oder Synthesizer aus den 80er Jahren aufschrauben, sehen Sie diese wunderschönen Keramikchips mit einem runden Glasfenster in der Mitte. Das ist ein EPROM. Die Idee war revolutionär: Man konnte den Chip unter eine UV-Lampe legen, ihn komplett löschen und neu beschreiben. Es dauerte eine halbe Stunde, aber es sparte enorm viel Geld bei der Entwicklung.

EEPROM (Electrically Erasable PROM)

- Löschen per elektrischem Signal statt UV-Licht
- In-System Programming: kein Auslöten nötig
- Byte-Ebene: einzelne Bytes gezielt löschen und neu beschreiben
- Kehrseite: große Zellen, geringe Dichte, hohe Kosten pro Megabyte
- Heute: kleine Konfigurationsdaten (MAC-Adresse, Kalibrierung)

Notizen

Das UV-Licht war unpraktisch. Das EEPROM löste dieses Problem, indem man durch geschicktes Anlegen von hohen elektrischen Spannungen die Elektronen wieder aus der Speicherzelle herausziehen konnte. Das Geniale: Man konnte gezielt ein einzelnes Byte löschen, während der Rest des Chips unangetastet blieb. Der Nachteil war jedoch die enorme Größe der einzelnen Zelle, weshalb EEPROMs niemals Gigabyte-Größen erreichten.

Flash-Speicher: Die Revolution der Blöcke

- Geburt des Flash (Toshiba, 1984): Kompromiss aus EPROM und EEPROM
- Löschen nicht byteweise, sondern blockweise „blitzartig“
- Architektur-Trick: Wegfall der byteweisen Löschlogik $\Rightarrow$ kleinere Zellen
- Resultat: hohe Speicherdichte zu niedrigen Preisen
- Dominante Technologie für Firmware und Massenspeicher (SSD, USB, Smartphones)

Notizen

In den 80er Jahren suchte die Industrie nach einem Weg, die Vorteile von EEPROMs (elektrisches Löschen) mit einer hohen Speicherdichte zu vereinen. Der Ingenieur Fujio Masuoka bei Toshiba erfand den Flash-Speicher. Sein Trick: Er warf die komplexe Verschaltung weg, die nötig war, um jedes Byte einzeln zu löschen. Flash kann nur blockweise gelöscht werden. Das sparte so viel Platz auf dem Silizium, dass Flash die Welt eroberte.

Das Floating-Gate (logische Sicht)

- Flash speichert Bits ohne Strom über Floating-Gate-Transistoren
- Zwei Gates: Control-Gate und isoliertes Floating-Gate
- Floating-Gate vollständig in Siliziumdioxid eingeschlossen
- Hohe Spannung kann Elektronen auf das Floating Gate bringen
- Die CPU sieht keine Elektronen, sondern eine verschobene Schwellspannung

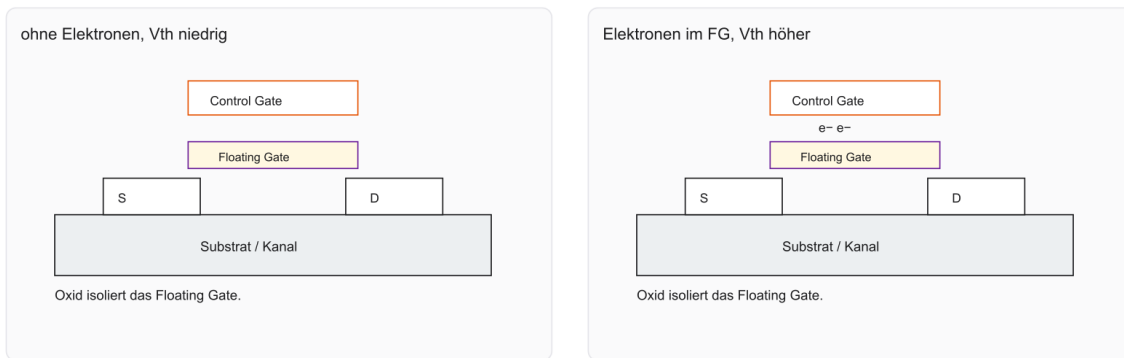


Abbildung 7: Querschnitt: Source, Drain, Kanal, Oxid, Floating Gate, Control Gate — mit und ohne Ladung.

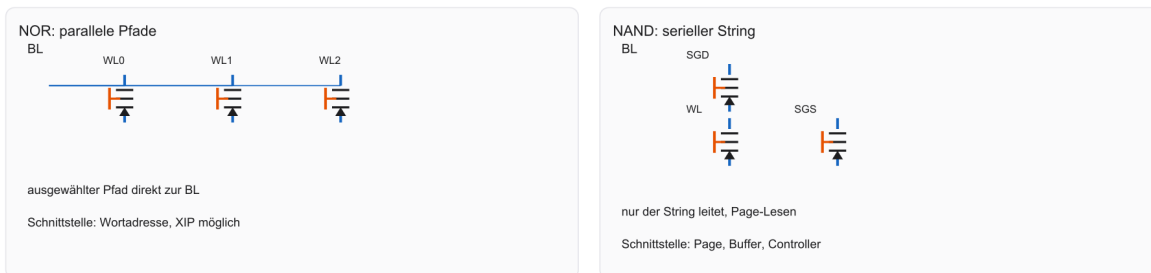
Notizen

Linkes Panel: leeres Floating Gate, niedrigere V_{th} . Rechtes Panel: gefangene Elektronen, höhere V_{th} . Charge-Trap-Flash ohne Floating Gate ist hier nicht gezeichnet.

NAND vs. NOR Flash

Flash spaltet sich in zwei Architekturen — je nach Verdrahtung der Floating-Gate-Zellen:

	NOR	NAND
Verdrahtung	parallel	seriell (Kette)
Zugriff	Byte/Wort (Random Access)	Seiten (Pages)
Typischer Einsatz	Mikrocontroller, XIP	SSD, Smartphones



Elektrische Vereinfachung. NOR: parallele Zellen. NAND: Serie plus Auswahltransistoren.

Deshalb XIP typisch bei NOR, block-/pageorientiert bei NAND — NAND ist kein Magnetband.

Abbildung 8: NOR: parallele Pfade zur Bitline. NAND: serieller String mit Auswahltransistoren. Darunter die unterschiedliche Schnittstelle.

Notizen

Die Verschaltung erklärt den Zugriff: NOR kann ein Wort adressieren, NAND liefert eine Page in einen Buffer. NAND ist pageorientiert, nicht „sequenziell wie ein Band“.

NOR-Flash & Execute-In-Place (XIP)

- XIP: Fetch holt Befehle direkt aus dem NOR-Flash (ohne vorherige RAM-Kopie des Codes)
- Fetch-Stage der RISC-V-Pipeline liest Code aus dem Flash
- Nachteil: geringere Dichte als NAND; Lesezugriff hat Waitstates, ist kein SRAM
- XIP heißt: der Fetch holt den Befehl aus dem NOR, nicht dass jede Adresse dieselbe Latenz hat

Notizen

NOR-Flash erlaubt wortweisen Lesezugriff, deshalb kann die Pipeline Befehle direkt daraus holen. Die Latenz liegt typisch über einem SRAM-Hit; „sofort wie SRAM“ wäre die falsche Ebene.

C-Code und der Flash-Speicher

- `const` erzwingt in C keine Flash-Platzierung
- Bei der hier betrachteten Embedded-Toolchain liegen *statische* Konstanten typischerweise in `.rodata`
- Welche Region `.rodata` trifft, entscheidet Linker-Skript und Startup, nicht das Schlüsselwort allein
- Ein normaler Store in eine nicht beschreibbare Region und ein konkreter RISC-V-Trap (`mcause`) sind zwei Ebenen
- Ohne Plattformmodell (kein `.repl` im Repo) wird hier kein bestimmter `mcause`-Wert behauptet

```
static const uint32_t table[] = {1, 2}; /* typisch .rodata */
```

Notizen

Ein AVR-`__flash`-Attribut ist ein Gegenbeispiel: erst ein herstellerspezifisches Attribut zwingt Daten in den Programmspeicher. In diesem Modul bleibt es bei der üblichen `.rodata`-Konvention.

NAND-Flash (Massenspeicher)

- Höchste Dichte durch serielle Verkettung der Zellen
- Page-orientiert: der Controller liefert eine Seite in einen Buffer, keine einzelne CPU-Adresse wie NOR
- Kein XIP: Code wird nach RAM kopiert
- Nicht rein sequenziell wie ein Magnetband — innerhalb der Page wählt der Controller



SLC-Modellkonvention: Programmieren 1→0, Erase zurück auf 1. Keine universelle Page-/Blockgröße.

Abbildung 9: SLC-Modell: gelöscht alles 1, Programmieren setzt ausgewählte Bits auf 0, Erase nur blockweise zurück.

Notizen

Ein gewöhnlicher Store kann weder die hohe Programmierspannung noch die Block-Löschung ersetzen. Page- und Blockgrößen stehen im Datenblatt, nicht als universelle Konstanten.

Folie

31 / 88

Die Flash-Krankheit: Wear and Tear

- Geheimnis: Flash verschleißt bei jedem Schreibvorgang
- Hohe Spannung beim Tunneln beschädigt die Oxid-Isolation
- Viele Schreib-/Löschzyklen ermüden das Oxid — die Zahl hängt vom Zelltyp ab, sie ist nicht unendlich und nicht für alle Technologien gleich

Notizen

Wenn Sie RAM beschreiben, hält die Zelle das ohne Programmier-Oxid. Flash altert mit Programmier-/Löschzyklen; wie viele Zyklen ein konkretes Medium verträgt, steht im Datenblatt.

Folie

32 / 88

Die Rettung: Wear Leveling (FTL)

- Wear Leveling: Verschleißausgleich über wechselnde physische Blöcke
- Logischer Sektor → jedes Mal anderer physischer Flash-Block
- Flash Translation Layer (FTL) übersetzt logisch ↔ physisch
- SD/SSD: FTL im Controller. Bare-Metal: Software muss das selbst leisten

Logischer Sektor 0 — drei Updates (schematisch)



FTL: logische Adresse → physischer Block. P/E-Zähler je Block steigen ungleich.

Wear Leveling wählt einen wenig genutzten Block. Garbage Collection ist optional und getrennt.

Update 1→4, Update 2→12 (4 ungültig), Update 3→27 (12 ungültig).

Abbildung 10: Dieselbe logische Adresse wandert über Updates auf neue physische Blöcke. Alte Kopien werden ungültig.

Notizen

Adressübersetzung, Verschleißausgleich und Garbage Collection sind drei Aufgaben. Die Abbildung zeigt nur die Zuordnung und ungültige Alt-Daten.

Memory Map: Wo liegt der Flash im SoC?

- In der 32-Bit Memory Map ist ein Block für Flash reserviert
- Beispieladressen stehen im Datenblatt der jeweiligen Plattform
- Im Repo gibt es kein `.repl` und kein Linker-Skript — die spätere Map ist ausdrücklich schematisch
- Boot per XIP nur, wenn der Fetch-Pfad den NOR-Flash wortweise lesen kann

Notizen

Lassen Sie uns den Bogen zurück zu MCT1 schlagen. Der Adressdecoder wählt den Flash-Bereich. Konkrete Basisadressen mischen wir nicht aus STM32- und SiFive-Beispielen.

Zusammenfassung: Non-Volatile Memory

- Festwertspeicher überlebt Stromausfall und hält Firmware/Boot-Code
- Evolution: Mask-ROM bis blockbasiertes Flash
- NOR-Flash: parallel, XIP, Standard in Mikrocontrollern
- NAND-Flash: seriell, dicht, Massenspeicher; braucht RAM zum Ausführen
- Flash verschleißt (Oxid-Schäden) und braucht Wear Leveling

Notizen

Damit haben wir die Basis unserer Speicherpyramide durchleuchtet. Wir wissen nun, wo unser Code liegt, wenn der Strom ausgeschaltet ist, und warum der Prozessor ihn direkt ausführen kann. Aber der Flash-Speicher hat ein Problem: Er ist viel zu langsam für die Arbeitsvariablen unserer Software und er verschleißt, wenn wir ihn ständig mit neuen Werten beschreiben würden.

Der Bedarf an Arbeitsspeicher

Wir brauchen Speicher, der:

- im Bereich der Pipeline nutzbar liest und schreibt
- sehr oft beschreibbar ist, ohne Flash-artigen Verschleiß

Antwort: volatiler Speicher (RAM).

Zwei Technologien: **SRAM** und **DRAM**.

Notizen

Für Variablen, Zwischenergebnisse der ALU und den Stack brauchen wir ein perfektes Kurzzeitgedächtnis. Als Nächstes öffnen wir den volatilen Arbeitsspeicher: zuerst SRAM, dann DRAM.

Fragen? — Non-Volatile

- Flash-Architektur?
- Execute-in-Place (XIP)?
- Wear Leveling?

Notizen

Bevor wir in die Transistorebene von SRAM und DRAM eintauchen: Ist klar, warum Mikrocontroller Code typischerweise aus NOR-Flash (XIP) ausführen und nicht aus NAND?

Kurze Pause

10 Minuten Pause.

Danach:

- SRAM (6-Transistor-Zelle)
- DRAM (1-Transistor-Zelle und Refresh)
- Spezialespeicher (FIFOs und Dual-Ported RAM)

Notizen

Atmen Sie kurz durch. Nach der Pause folgt die Physik des Arbeitsspeichers.

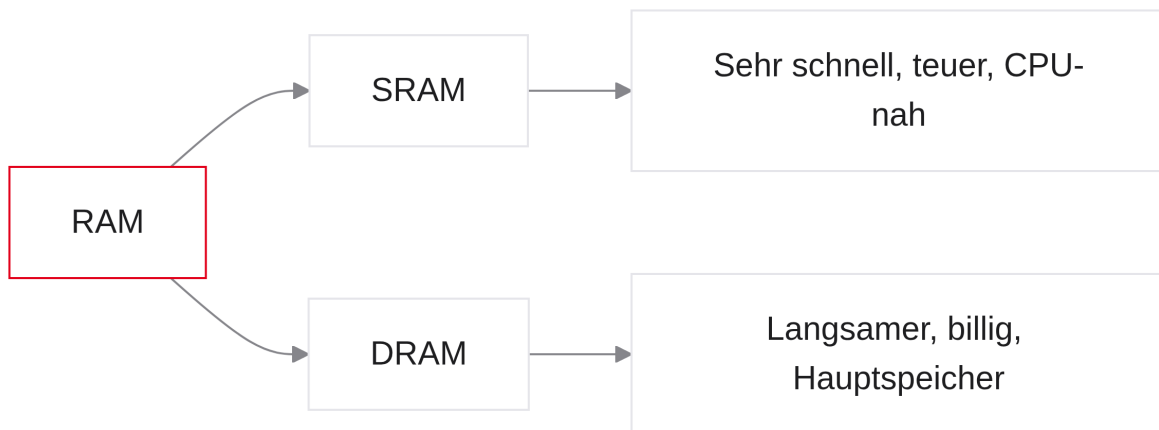
Volatiler Speicher: RAM

- Arbeitsspeicher: Kurzzeitgedächtnis des Prozessors
- Volatil: Daten weg ohne Versorgungsspannung
- Wahlfreier Zugriff: jede Adresse gleich schnell
- Zwei Architekturen: SRAM (Static RAM) und DRAM (Dynamic RAM)
- Gleicher Zweck, grundlegend andere Physik

Notizen

Nachdem wir geklärt haben, wie Code den Stromausfall überlebt, widmen wir uns den flüchtigen Speichern. Wenn Sie in C ein Array anlegen oder eine Variable definieren, leben diese im RAM. Damit der Prozessor nicht unnötig wartet, muss dieser Speicher schnell sein. Wir unterscheiden statischen und dynamischen RAM.

RAM: SRAM und DRAM



Notizen

Schematisch: SRAM sitzt nahe an der CPU (Caches, Register), DRAM bildet den großen Hauptspeicher. Beides speichert Bits — mit völlig unterschiedlichen Bauteilen.

SRAM — Prinzip

- Statisch: Wert bleibt ohne Auffrischen, solange Strom anliegt
- Kein Refresh nötig (im Gegensatz zu DRAM)
- Schnellste verfügbare Speichertechnologie
- Latenz oft unter 1 ns
- Kehrseite: geringe Dichte — viele Transistoren pro Zelle

Notizen

„Statisch“ bedeutet hier nicht, dass Daten bei Stromausfall erhalten bleiben, sondern dass das Bit im Transistornetzwerk stabil liegt, solange die Betriebsspannung da ist. SRAM ist das Rückgrat schneller Caches und Register.

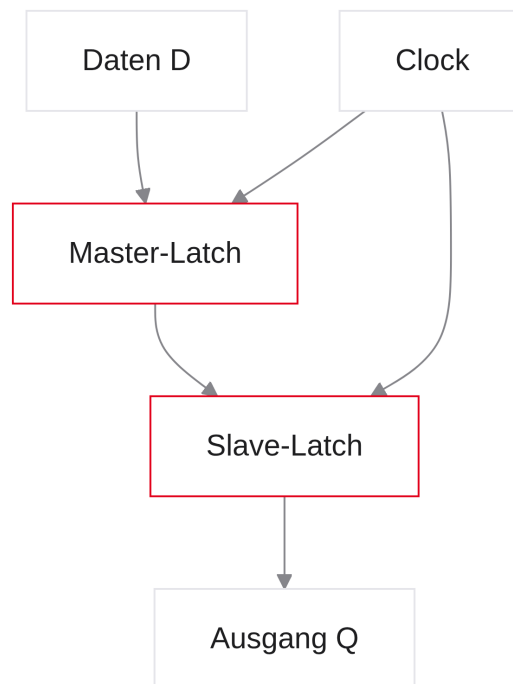
Refresher: Das D-Flip-Flop (MCT1)

- Register in MCT1: D-Flip-Flops
- Master-Slave-D-FF: oft über 20 Transistoren
- 1 MB mit Flip-Flops: Hunderte Millionen Transistoren
- Zu viel Fläche und Abwärme
- Lösung: Design auf das physikalische Minimum eindampfen

Notizen

Für 32 Register sind Flip-Flops vertretbar. Für Megabyte-Caches im Die brauchen wir eine radikal kleinere Zelle.

D-Flip-Flop: Master und Slave



Notizen

Schematisch: Zwei Latches hintereinander. Funktional stark — flächenmäßig zu teuer für großen SRAM.

Die 6T-SRAM-Zelle

- Klassisches SRAM-Bit: exakt 6 Transistoren (6T)
- Vier Transistoren: zwei kreuzgekoppelte Inverter
- Zwei Access-Transistoren als „Türsteher“
- Wordline (WL): öffnet die Zelle
- Bitlines (BL / $\overline{\text{BL}}$): Daten hinein und heraus

Notizen

Statt rund 20 Transistoren bleiben sechs: zwei Inverter im Ring und zwei Schalter zu den Bitlines. Eine der am häufigsten gebauten Zellen der Welt.

6T-Zelle: Überblick

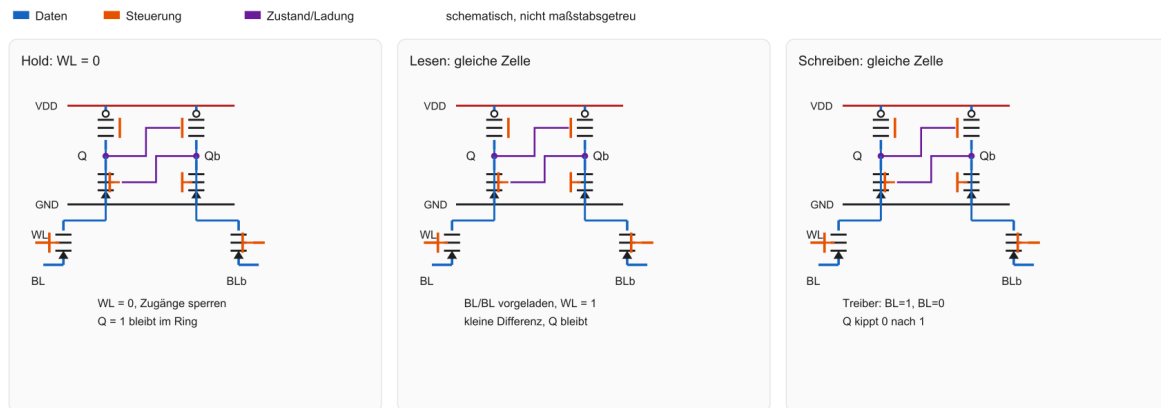


Abbildung 11: Dieselbe 6T-Geometrie: vier Inverter-Transistoren, zwei Access-Transistoren, Q/Q , BL/BL , WL, VDD, GND. Hold, Lesen und Schreiben ändern nur die Signale.

Notizen

Beide Rückkopplungen sind eingezeichnet: Q steuert den rechten Inverter, Q den linken. Die Access-Transistoren hängen an WL.

Der kreuzgekoppelte Inverter-Ring

- Bit speichern ohne Taktsignal
- Inverter: $1 \rightarrow 0, 0 \rightarrow 1$
- Ausgang von Inverter 1 $\rightarrow$ Eingang von Inverter 2 und zurück
- Stabiler Ring: Zustand hält sich selbst
- Bistabilität: Gleichgewicht, solange Strom anliegt

Notizen

Zwei NOT-Gatter im Kreis: Die linke 1 erzwingt rechts 0, die 0 erzwingt links wieder 1. Das System stabilisiert sich selbst — angetrieben von der Versorgungsspannung.

Zustand halten (Hold State)

- Wordline = 0 V: Hold State
- Access-Transistoren sperren
- Zelle von den Bitlines isoliert
- Strom nur im Inverter-Ring
- Ruhestrom theoretisch sehr gering

Notizen

Die meiste Zeit bleibt die Zelle im Hold State: Türen zu, Bit im Ring gefangen. Aktivität auf den Bitlines berührt die Zelle nicht.

Hold State — Schema

Linkes Panel der 6T-Abbildung: WL = 0, Access-Transistoren sperren, der Ring bleibt geschlossen.

Notizen

Gleiche Geometrie wie Lesen und Schreiben. Nur WL und die Bitline-Treiber ändern sich.

SRAM Lesen

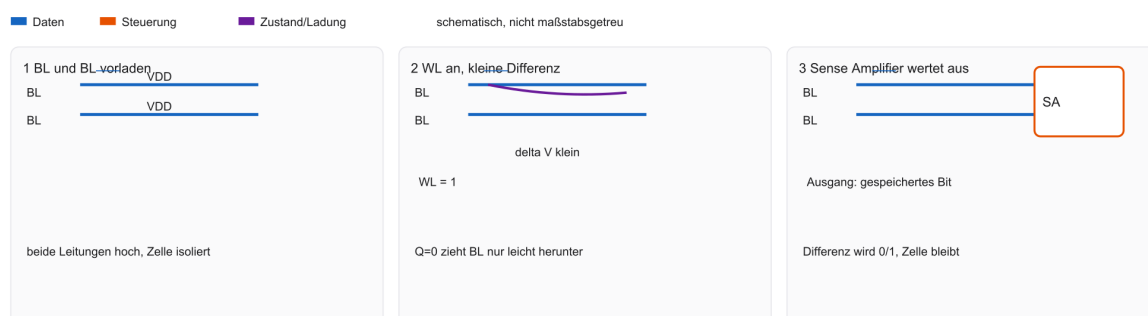


Abbildung 12: Drei Panels: Vorladen, WL aktiv mit kleiner Bitline-Differenz, Sense Amplifier. Der Zellzustand bleibt erhalten.

- Die Zelle zieht eine Leitung nur leicht; sie wird im vorgesehenen Lesen nicht umgeschrieben

Notizen

Panel 1: BL und BL auf hoher Spannung. Panel 2: WL = 1, kleine Differenz. Panel 3: der Leseverstärker digitalisiert, Q bleibt.

Folie

49 / 88

SRAM Schreiben



Abbildung 13: Beispiel Q von 0 nach 1: komplementäre Treiber, WL an, Zustand kippt, WL wieder aus.

- Die Schreibtreiber müssen die Rückkopplung überwinden; eine feste Transistor-Größenregel gilt nicht für jede Bibliothek

Notizen

Panel 1 setzt BL und BL gegensinnig. Panel 2 öffnet WL. Panel 3 isoliert die Zelle mit dem neuen Wert.

Folie

50 / 88

Vorteile von SRAM

- Sehr kurze Zugriffszeit auf Makro-Ebene — nicht mit einem kompletten DRAM-Burst vergleichen
- Kein Refresh der Zelle
- Einfache Schnittstelle: Adresse anlegen, Daten abholen
- Geringer Stromverbrauch bei Inaktivität
- Integration im selben Logic-Prozess wie die ALU

Notizen

SRAM bremst die Pipeline kaum und lässt sich im gleichen Silizium-Prozess wie die Rechenlogik fertigen — ideal für Caches.

Nachteile von SRAM

- Geringe Datendichte: 6 Transistoren pro Bit brauchen Fläche
- Hohe Kosten: Chipfläche ist teuer
- Kapazitäts-Limit: selten mehr als einige Dutzend MB SRAM als Cache
- Ungeeignet für Massenspeicher (Laptop mit 16 GB SRAM wäre unrealistisch)

Notizen

Sechs Transistoren pro Bit reichen für Caches, nicht für Gigabyte-Hauptspeicher. Deshalb bleibt DRAM unverzichtbar.

Die Leckströme (Leakage)

- Moderne Nodes (z. B. 3 nm): Transistoren sperren nicht mehr perfekt
- Static Leakage auch im Hold State (Quantentunneling)
- Folge: Strom und Wärme nur zum Datenhalten
- Mobile SoCs: Power Gating ganzer SRAM-Blöcke im Standby

Notizen

Bei wenigen Nanometern „lecken“ Transistoren. Große SRAM-Caches können im Standby spürbar Energie ziehen — deshalb Power Gating in Smartphones.

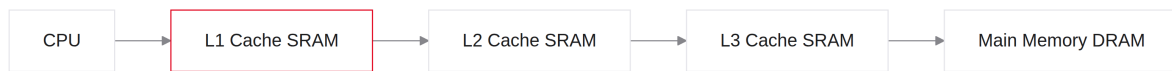
Einsatzgebiet von SRAM im SoC

- Registerfiles können aus Flip-Flops, Latches oder speziellen SRAM-Strukturen bestehen
- Ein RISC-V-Register ist nicht pauschal eine Standard-6T-Zelle
- L1/L2 sind häufig SRAM-Makros; die Latenz eines Cache-Hits ist nicht die Zellzeit allein

Notizen

SRAM sitzt an der Spitze der Speicherpyramide: Register und Caches. Nur bei Miss geht es weiter zum DRAM.

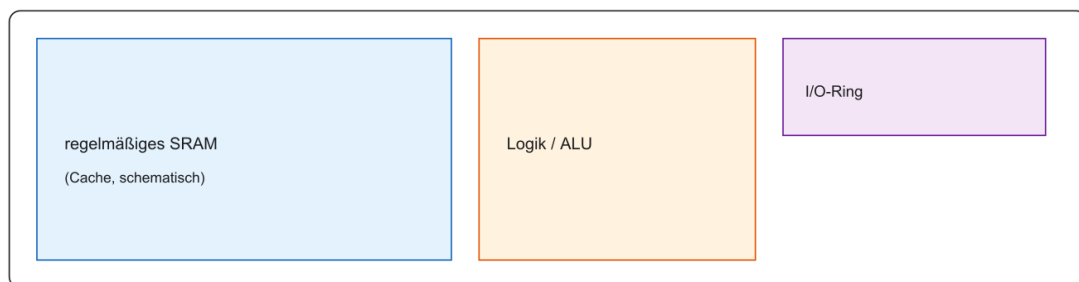
SRAM in der Hierarchie



Notizen

Schematisch: Jede Stufe fragt die nächste, wenn Daten fehlen. Moderne Kerne sind oft von großen SRAM-Caches umgeben.

Schematischer Floorplan



Schematischer Floorplan, keine Mikroskopaufnahme.

Abbildung 14: Regelmäßige SRAM-Fläche, unregelmäßigere Logik, I/O-Ring. Keine Mikroskopaufnahme.

Notizen

Ohne belegbare, passend lizenzierte Die-Aufnahme bleibt es bei diesem Floorplan. Titel bewusst nicht „Mikroskop“.

Zusammenfassung SRAM

- Typ: volatiler Arbeitsspeicher
- Zelle: 6T (Inverter-Ring)
- Vorteile: kein Refresh, sehr schnell
- Nachteile: flächenintensiv, Leckströme bei kleinen Nodes
- Einsatz: Caches und CPU-naher Speicher im SoC

Notizen

SRAM ist teuer und schnell — unverzichtbar für die Pipeline. Für den großen Hauptspeicher brauchen wir dichtere Technik: DRAM.

Fragen? — SRAM

- Aufbau der 6T-Zelle?
- Schreib- oder Lesevorgang?
- Warum kein Refresh bei SRAM?

Notizen

Ist das Konzept des selbst-erhaltenden Inverter-Rings klar, bevor wir zu DRAM wechseln?

Volatiler Speicher: DRAM

- Ziel: Gigabyte Arbeitsspeicher zu akzeptablen Kosten
- Weg: Bauteile pro Bit radikal reduzieren
- Ergebnis: Dynamic RAM (DRAM)
- Dynamisch: Zelle hält den Zustand nicht allein — Ladung geht in Millisekunden verloren
- Zelle einfach, Controller komplex

Notizen

Ein 16-GB-Riegel ist DRAM. Maximale Dichte erfordert komplexe Controller, weil die Zellen ihre Ladung nicht dauerhaft halten.

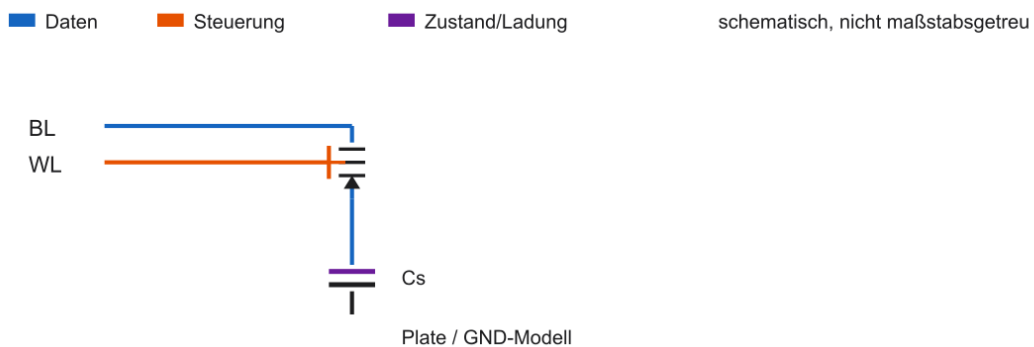
Die 1T1C-Zelle

- Flip-Flop: ≈ 20 Transistoren; SRAM: 6; DRAM: 1 Transistor + 1 Kondensator
- Kondensator: speichert Ladung („Eimer“)
- Transistor: Schalter zum Füllen/Leeren
- Weniger Bauteile pro Bit sind kaum möglich

Notizen

1T1C ist der Dichte-Kompromiss: Ein Transistor, ein Kondensator. Maximale Packungsdichte — um den Preis von Refresh und komplexer Ansteuerung.

1T1C — Schema



WL am Gate, BL am Drain, Speicherknoten am Source.

Abbildung 15: Access-Transistor: WL am Gate, BL als Datenpfad, Kondensator gegen eine Plate-Elektrode (hier als GND-Modell).

Notizen

Der Kondensator hängt nicht „irgendwo an Masse“ ohne Kennzeichnung: die Gegenelektrode ist das Plate-Potential, in der Zeichnung vereinfacht nach GND geführt.

Kondensator als Speicher

- Geladen (hohe Spannung) → logische 1
- Entladen (≈ 0 V) → logische 0
- Kapazitäten im Femtofarad-Bereich — einige zehntausend bis hunderttausend Elektronen
- 3D-Fertigung: Trench-/Stack-Kondensatoren für Dichte

Notizen

Um Kapazität bei kleinem Fußabdruck zu gewinnen, werden Kondensatoren als Gräben oder Stapelstrukturen gefertigt — hochpräzise Halbleitertechnik.

Was bedeutet „dynamisch“?

- Die Spannung einer gespeicherten 1 sinkt durch Leckstrom
- Das Refresh-Fenster ist geräte- und temperaturabhängig; 64 ms ist nur ein bekanntes Beispiel, keine Lebensdauer jedes Bits

Notizen

Das „D“ steht für dynamisch: ohne organisiertes Auffrischen von Zeilen und Bänken geht die Ladung verloren. Nicht jede Zelle ist nach exakt 64 ms leer, und der ganze Riegel steht nicht für 64 ms still.

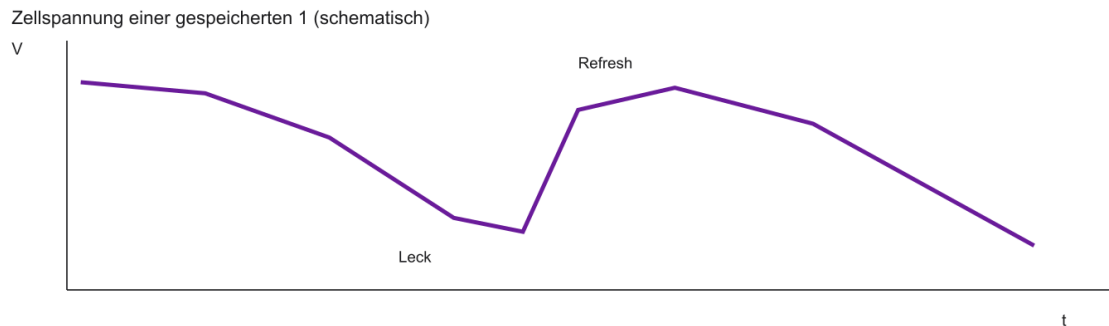
Der Refresh-Zyklus

- Refresh läuft über Zeilen und Bänke, nicht als ein Schlag für den gesamten Speicher
- Ob und wie lange Zugriffe dabei warten, ist geräteabhängig

Notizen

Der Controller liest eine Zeile und stellt sie wieder her, bevor das Fenster abläuft. 64 ms bleibt ein Beispiel, kein festes Gesetz.

Refresh — Schema



64 ms nur als geräte- und temperaturabhängiges Beispiel-Fenster, keine feste Bit-Lebensdauer.

Abbildung 16: Spannung über der Zeit: Ladungsverlust, rechtzeitiges Auffrischen, wiederhergestellter Pegel.

Notizen

Die Kurve ist qualitativ. Der Knick ist ein Refresh, kein Messpunkt.

DRAM-Lesen ist destruktiv

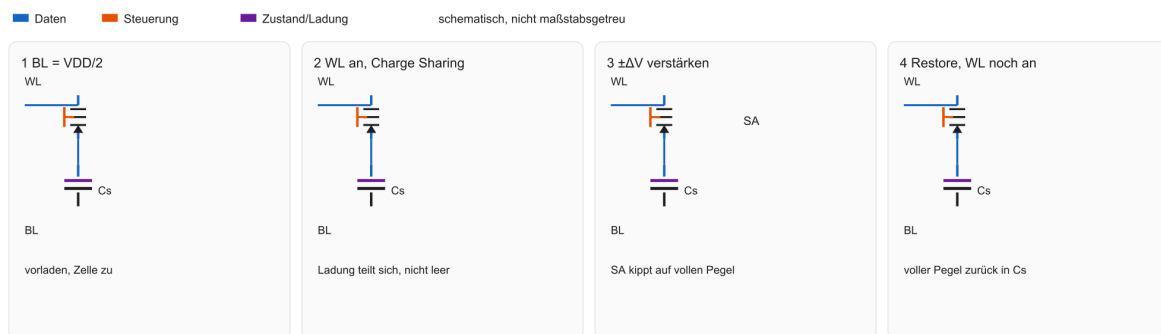


Abbildung 17: Vier Panels: BL auf VDD/2, Charge Sharing, Verstärkung, Restore bei noch aktiver WL. Der Kondensator läuft nicht einfach leer.

- Charge Sharing verschiebt BL und Zelle auf einen gemeinsamen Zwischenpegel
- Der Sense Amplifier treibt danach den vollen Pegel zurück in die Zelle

Notizen

Panel 2 ist keine Entleerung auf 0. Panel 4 stellt den Zellenpegel wieder her, solange WL aktiv ist.

Sense Amplifiers

- Panels 3 und 4 der Lese-Abbildung: kleine Abweichung $\pm\Delta V$ wird zum vollen Pegel
- Derselbe volle Pegel schreibt die Zelle zurück (Restore)
- Refresh einer Zeile nutzt denselben Mechanismus

Notizen

Ohne Leseverstärker wäre die geteilte Ladung nicht als Bit entscheidbar. Der Restore ist Teil des Lesens, nicht ein späterer Store der CPU.

DRAM Schreiben

- Bitline hart auf High oder Low treiben
- Wordline öffnet den Access-Transistor
- Kondensator füllt/entleert sich schnell
- Wordline zu — Bit gefangen (bis zum nächsten Refresh)

Notizen

Schreiben ist der einfachere Teil: Treiber setzen den Pegel, Transistor verbindet, Kondensator übernimmt.

Adress-Matrix und Multiplexing

- Zellen in Zeilen und Spalten, oft mehrere Bänke
- Die Adresse zerfällt in Bank, Row, Column und Byte-Offset — die Breiten sind nicht zwei gleiche Hälften

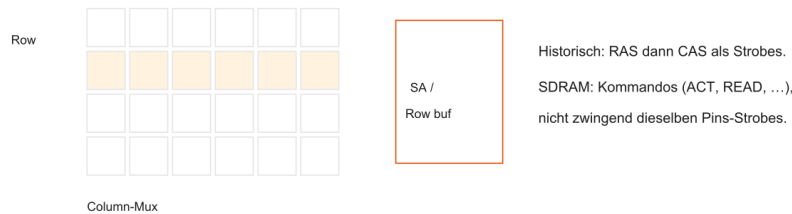
Notizen

Multiplexing spart Pins, weil Zeile und Spalte nacheinander kommen. Die Bitaufteilung steht im Speicher-Datenblatt, nicht in der CPU-Wortbreite.

Multiplexing — Schema

Illustrative Adresse, nicht zwei gleiche Hälften einer 32-Bit-Adresse

Bank | Row | Column | Byte-Offset (Breiten geräteabhängig)



Zeile wird aktiviert, Spalte wählt aus dem Row Buffer. Orange: ausgewählte Zeile.

Abbildung 18: Kleine Matrix, Zeilendecoder-Zeile markiert, Spaltenauswahl, Sense Amplifier / Row Buffer. Adressfelder nur illustrativ.

Notizen

Die orange Zeile ist aktiv. Der Row Buffer hält sie, die Spalte wählt daraus. Keine 16+16-Aufteilung einer 32-Bit-Adresse.

RAS und CAS

- Historisch: RAS und CAS als Strobes für Zeile und Spalte
- Moderne SDRAM-Schnittstellen nutzen Kommandos (Activate, Read, Precharge), nicht zwingend dieselben Strobes
- Die Abbildung der vorigen Folie trennt Matrix und diese Schnittstellen-Ebene

Notizen

BIOS-„CAS Latency“ beschreibt ein Speicherkommando-Timing, nicht automatisch den klassischen RAS/CAS-Pin von 1980.

Warum DRAM langsamer ist als SRAM

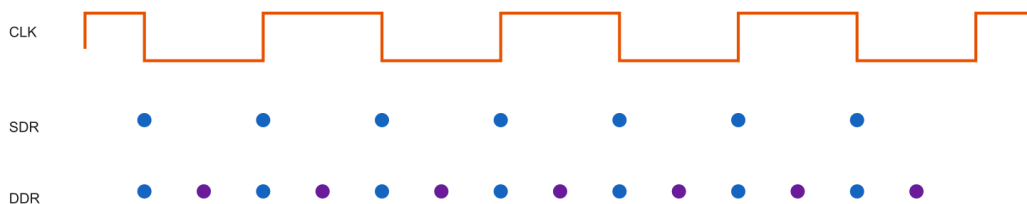
- Adress-Multiplexing (Row, dann Column)
- Analoges Auslesen und Sense Amplifier
- Destruktives Lesen inkl. Zurückschreiben / Precharge
- Mögliche Wartezeiten während Refresh

Notizen

Die Latenz folgt aus Physik und Organisation — nicht aus „Böswilligkeit“. Der Inverter-Ring des SRAM antwortet deutlich direkter.

SDRAM und DDR

Gleicher Schnittstellentakt — SDR vs. DDR (schematisch)



Punkte: gültige Datenübernahme. DDR nutzt beide Flanken.

Bandbreite etwa doppelt — die Zeilen-Latenz halbiert sich dadurch nicht.

Abbildung 19: Derselbe Takt. SDR übernimmt auf einer Flanke, DDR auf beiden. Die Schnittstellen-Bandbreite steigt, die Zeilenlatenz nicht automatisch auf die Hälfte.

- LPDDR ist eine energiesparende Familie derselben Zellidee

Notizen

Die Punkte im Diagramm sind Übernahmezeitpunkte, keine Latenzpfeile. DDR verdoppelt die Transfers pro Takt an der Schnittstelle.

Einsatzgebiet von DRAM im SoC

- Meist nicht auf dem Logic-Die (anderer Prozess)
- Separater Chip oder Package-on-Package
- Main Memory: OS, Heap (`malloc`), große Arrays
- Kleine MCUs: oft nur interner SRAM, kein DRAM

Notizen

Große SoCs (z. B. Linux-fähig) brauchen DRAM. Kleine Sensor-MCUs kommen oft mit Kilobyte-SRAM aus.

Vergleich: SRAM vs. DRAM

	SRAM	DRAM
Zelle	6T Inverter-Ring	1T1C
Datenerhalt	stabil (mit Strom)	Refresh nötig
Geschwindigkeit	kurz auf SRAM-Makro-Ebene	länger für einen vollen DRAM-Zugriff
Dichte	gering	sehr hoch
Einsatz	Caches; Register je nach Implementierung	Hauptspeicher
Kosten / Bit	hoch	niedrig

Notizen

Kein „besser“ oder „schlechter“ — beide sind für ihre Ebene in der Hierarchie optimiert. SRAM ist der schnelle Kurier, DRAM das Containerschiff.

Spickzettel SRAM / DRAM

Eigenschaft	SRAM	DRAM
Zellen-Architektur	6 Transistoren	1T + 1C
Datenerhalt	stabil (Strom an)	flüchtig (Refresh)
Geschwindigkeit	Zell-/Makro-Ebene, nicht Cache-Hit	voller Zugriff inkl. Zeile/Spalte
Speicherdichte	gering	extrem hoch
Einsatz im SoC	häufig Caches	Main Memory, oft off-chip
Kosten pro Bit	hoch	niedrig

Angaben schematisch — typische Größenordnungen.

Notizen

Diese Dimensionen reichen oft für Klausur und Interview: Cache vs. Hauptspeicher physikalisch begründet.

Fragen? — DRAM

- Kondensator-Prinzip?
- Destruktives Lesen?
- Refresh-Zyklus klar?

Notizen

Ist verständlich, warum Refresh und Multiplexing die Pipeline verzögern können?

Spezialspeicher im SoC

- Neben Cache und Main Memory: spezialisierte RAM-Zellen
- Problem: CPU und Peripherie mit unterschiedlichen Takten
- Wie Daten austauschen ohne gegenseitiges Blockieren?
- Lösungen: Dual-Ported RAM (DPRAM) und FIFOs

Notizen

Manchmal müssen zwei Module gleichzeitig auf denselben Puffer zugreifen. Normales Single-Port-RAM reicht dann nicht.

Dual-Ported RAM (DPRAM)

- Zwei Ports mit eigener Adresse, eigenen Daten und eigener Steuerung
- Gleichzeitig an verschiedenen Adressen möglich
- Die Transistorzahl hängt von der konkreten Zelltopologie ab; „8T“ ist keine allgemeine Definition

Notizen

Beschreiben Sie die Ports, nicht eine feste Transistorzahl.

DPRAM — Schema



Verschiedene Adressen: konfliktfrei. Dieselbe Adresse: Verhalten implementierungsabhängig.

Keine feste „8T“-Zelle. Arbitration nur, wenn der Baustein sie vorsieht.

Abbildung 20: Port A und Port B jeweils mit Adresse, Daten und Steuerung.

Notizen

Die Leitungen sind beschriftet. Ein einzelner Kasten ohne Ports würde die Lernfrage nicht beantworten.

Folie

80 / 88

Kollisionsvermeidung im DPRAM

- Verschiedene Adressen: beide Ports arbeiten
- Dieselbe Adresse: Ergebnis und Arbitration sind implementierungsabhängig
- Ein Busy-Flag oder Stall ist eine mögliche, keine universelle Lösung

Notizen

Die DPRAM-Abbildung gilt für beide Fälle. Kollisionsverhalten steht im Datenblatt der Zelle, nicht in einer allgemeinen Regel.

Folie

81 / 88

FIFO-Puffer

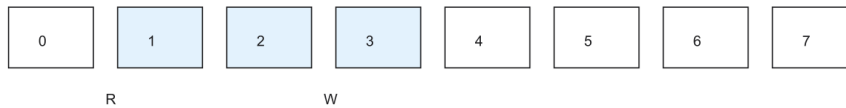
- First-In-First-Out: Warteschlange
- Keine expliziten Adressen für Push/Pop
- Hardware: SRAM mit Write-/Read-Pointer (Ringpuffer)

Notizen

FIFO ist die übliche Warteschlange in *einem* Taktbereich. Zwischen zwei Takten braucht es eine asynchrone FIFO mit Synchronisation, nicht nur den Ring.

FIFO — Schema

Ringpuffer, ein Taktbereich. Async-FIFO braucht extra Synchronisation.



Push erhöht W, Pop erhöht R, beide mit Wraparound.

Empty: $R=W$. Full: nächster W würde R treffen (Konvention des Zählers).

UART: Watermark löst einen Interrupt aus, nicht ein Interrupt pro Bit.

Abbildung 21: Ring mit Write- und Read-Pointer, belegten Einträgen, Wraparound. Empty und Full als Zählerkonvention.

Notizen

Ein gewöhnlicher Ringpuffer sitzt in einem Taktbereich. Eine asynchrone FIFO zwischen zwei Takten braucht zusätzliche Synchronisation; der Ring allein löst Clock-Domain-Crossing nicht.

Hardware-FIFOs für I/O

- Übliche UART: ohne FIFO ein Interrupt pro empfangenem Zeichen, nicht pro Bit
- Mit FIFO: die CPU wird erst am Watermark geweckt
- Stehen Sender und Empfänger in verschiedenen Takten, ist die FIFO asynchron und braucht Synchronisationslogik

Notizen

Das Watermark steht in der Ringpuffer-Abbildung als Idee: nicht jedes Byte unterbricht. Bit-Interrupts wären ein unübliches Bit-Banging, keine Hardware-UART.

Brückenschlag zur Software

- CPU sieht keinen Kondensator — nur linearen 32-Bit-Adressraum
- Adress-Decoder lenkt Bereiche an physische Bausteine
- Beispiel: Pointer auf Flash vs. Pointer auf SRAM/DRAM

Notizen

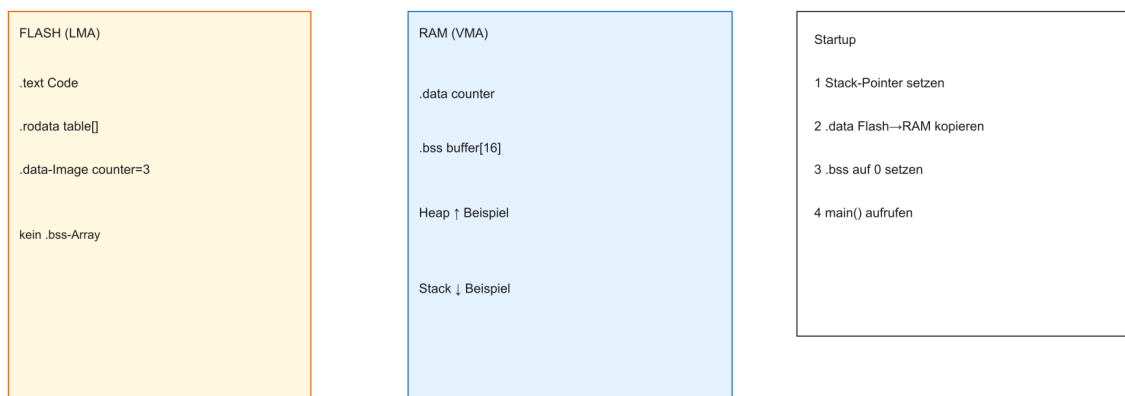
Memory-Mapping verbindet C-Pointer mit der Physik. Der Compiler kennt Transistoren nicht — der Decoder schon.

Memory Map eines Mikrocontrollers

Schematisches Beispiel, keine Plattformadressen aus dem Repo:

```
static const uint32_t table[] = {1, 2};
uint32_t counter = 3;
uint32_t buffer[16];
```

Schematische MCU-Map — keine STM32-/SiFive-Adressen



static const uint32_t table[]={1,2}; uint32_t counter=3; uint32_t buffer[16];

Abbildung 22: C-Objekte, ELF-Sections, Flash-LMA und RAM-VMA. Startup kopiert .data und nullt .bss.

Notizen

table liegt in .rodata im Flash. counter hat ein Initialbild im Flash (LMA) und die Laufzeitvariable im RAM (VMA). buffer liegt nur im RAM. Heap wächst im Beispiel nach oben, der Stack nach unten — das ist eine Konvention, kein Standard der ISA.

Das Linker-Skript

Ein MEMORY-Block allein legt Sections nicht ab. Typische Toolchain (schematisch, GNU ld: VMA = Laufzeit, LMA = Ladeadresse):

```
MEMORY {
  FLASH (rx) : ORIGIN = 0x00000000, LENGTH = 256K
  RAM (rwx)  : ORIGIN = 0x10000000, LENGTH = 64K
}
SECTIONS {
  .text : { *(.text*) } > FLASH
  .rodata : { *(.rodata*) } > FLASH
  .data : {
    _data_start = .;
    *(.data*)
    _data_end = .;
  } > RAM AT> FLASH
  .bss : {
    _bss_start = .;
    *(.bss*)
    _bss_end = .;
  } > RAM
}
```

Startup kopiert `_data_start` aus der LMA im Flash und setzt `.bss` auf 0. Die Adressen sind Platzhalter, kein STM32- und kein SiFive-Layout.

Notizen

> RAM AT> FLASH trennt VMA und LMA von `.data`. `.bss` hat kein Initialbild im Flash. Quelle der Begriffe: GNU ld, Basic Script Concepts.

Ausblick Labor (Renode)

Geplantes Laborziel (Details folgen mit dem Übungsblatt):

- C-Projekt für einen RISC-V-SoC kompilieren
- Linker-Skript und Memory Map untersuchen
- Binary in Renode laden
- Speicherinhalte beobachten

Notizen

Sobald das Übungsblatt steht, verifizieren Sie die Konzepte im Simulator — nicht nur „Kompilieren und hoffen“.

Zusammenfassung Block 1

- Speicherphysik: Kompromisse aus Kosten, Geschwindigkeit, Dichte
- Flash (non-volatil): Firmware; NOR oft mit XIP in MCUs
- SRAM (6T): schnell, teuer — Caches und Register
- DRAM (1T1C): dicht, Refresh — Hauptspeicher
- Linker-Skript verteilt Software auf physische Zonen

Notizen

Damit ist Block 1 geschlossen. In Block 2 (Vorlesung 2) sehen wir, wie Caches die DRAM-Latenz verbergen. Vielen Dank für Ihre Aufmerksamkeit.